



µLab worksheet • Deploying Spark on your own cluster

Group _____ Date _____ Time _____

Driver (types) _____ Mapper (looks up) _____ Referee (demos) _____

Hand this in at the end of the round. The referee presents this work in 90 seconds at the caucus, so the referee has to follow it, not just watch it.

1 • GIVE THE CLUSTER A FOLDER IT CAN READ • STEP 1

A container can only see the folders it was handed at the moment it was created. µLab 1 handed the cluster exactly one, `kube`, so that it had somewhere to write its credentials. There is no command that adds a second folder afterwards: the only way is to throw the container away and make it again. So make an empty folder now and hand it over now, while rebuilding is free.

1 ☐ Make the folder, then rebuild the cluster around it.

Work in the same folder you started the cluster in during µLab 1, the one with `kube` in it.

```
mkdir lab
```

Do not skip this line. If `lab` does not exist when the next command runs, Docker creates it for you and it comes out owned by `root`. Everything you try to write into it afterwards then fails with `Permission denied`, and the only cure is to delete it with `sudo` and start again.

Now replace the cluster.

The first line deletes the µLab 1 cluster and everything on it, including the whale. That is intended: nothing from µLab 1 is needed again.

```
docker rm -f k8s
docker run -d --name k8s --privileged --tmpfs /run --tmpfs /var/run \
  -p 6443:6443 -v "$PWD/kube:/output" -v "$PWD/lab:/lab" \
  rancher/k3s:v1.36.4-k3s1 server --disable=traefik \
  --write-kubeconfig /output/config --write-kubeconfig-mode 644
```

This is the µLab 1 command with **one argument added**. The table repeats the rest in one line each so that nothing on this sheet is typed unexplained.

<code>rm -f k8s</code>	Delete the container named <code>k8s</code> . The <code>-f</code> kills it first if it is still running.
<code>-d</code>	Detached: the cluster keeps running after your prompt comes back.
<code>--name k8s</code>	Give it a name, so later commands can say <code>k8s</code> instead of an id.
<code>--privileged</code>	Kubernetes runs containers, so it needs the kernel privileges a container is normally denied.
<code>--tmpfs /run</code>	A small in-memory filesystem. <code>k3s</code> writes sockets there and they must not survive a restart.
<code>--tmpfs /var/run</code>	The same, for the second conventional location.
<code>-p 6443:6443</code>	Publish the API port to your laptop, so a client outside the container can reach it.
<code>-v "\$PWD/kube:/output"</code>	Bind-mount: the <code>kube</code> folder here appears inside at <code>/output</code> , which is where the credentials file gets written.
<code>-v "\$PWD/lab:/lab"</code>	The new one. The <code>lab</code> folder here appears inside the cluster at <code>/lab</code> . It is empty, and it stays yours: a bind mount is the same folder seen twice, not a copy, so whatever you put in it later is visible inside at once.
<code>rancher/k3s:...</code>	The image: a whole Kubernetes in one binary. Pinned to <code>v1.36.4-k3s1</code> so every laptop runs the same version.
<code>server</code>	Everything after the image name is handed to the program inside. <code>server</code> means be the cluster, not join one.
<code>--disable=traefik</code>	Skip the bundled web router. Nothing here serves web traffic, and it takes time to start.
<code>--write-kubeconfig ...</code>	Where to write the credentials file, and with which permissions. <code>644</code> makes it readable by you rather than only by root.

Then get a client. It is the µLab 1 line, unchanged.

Linux users who installed `kubectl` in µLab 1 keep using it and skip this box. Nothing about the client changes this round: it talks to the cluster over the network and never touches `lab`.

```
docker run --rm -it --network container:k8s \
  -v "$PWD/kube:/kube:ro" -e KUBECONFIG=/kube/config \
  --entrypoint sh alpine/kubectl
```

<code>--rm -it</code>	Disposable, and interactive: a shell you can type many commands into, deleted when you leave.
<code>--network container:k8s</code>	Join the cluster container's network instead of getting one. That is why <code>127.0.0.1:6443</code> works in here.
<code>-v "\$PWD/kube:/kube:ro"</code>	The credentials, read-only. A client has no business editing them.
<code>-e KUBECONFIG=/kube/config</code>	Which cluster to talk to. The path is the one inside this container , not on your laptop.

```
--entrypoint sh
alpine/kubect1
```

Run a shell instead of the single `kubect1` the image is built for.
The image: `kubect1` and a small Alpine base, from Docker Hub.

Check the cluster answers. The `NAME` is the new container's id, so it is **not** the one `µLab 1` showed: this is a different cluster.

```
kubect1 get nodes
```

`NAME` _____ `STATUS` _____

2 • SOME DEFINITIONS

Everything this round asks you to explain can be answered from this block. It is worth reading once before step 2 rather than hunting through it afterwards.

image store	The place a container runtime keeps the images it can start. Your laptop's Docker has one. The cluster runs a different runtime, containerd, with a store of its own, and neither can read the other's files. They use the same names, which is what makes the confusion easy. Panel 4 lists both.
manifest	A file describing objects you want to exist, rather than commands that create them. <code>µLab 1</code> typed <code>kubect1 create deployment</code> , one object per command. A manifest lists several, and <code>kubect1 apply</code> makes the cluster match it: creating what is missing, changing what differs, leaving the rest. Running it twice changes nothing the second time.
driver	The program that runs your Spark code. It decides what work exists, hands the pieces out, and collects the answers. There is exactly one per job, and when it stops the job is over.
executor	A process that does pieces of work for one driver. It is started for a job and exits with it. An executor holds a number of cores, and runs one piece of work per core at a time.
Spark master	The process that hands out the cluster's cores. It runs nothing itself. Drivers ask it for resources, it decides which machines will supply them, and it keeps a list of who is available. Not to be confused with the Kubernetes control plane, which hands out pods. This one hands out cores, and it is a pod itself.
Spark worker	A process that offers cores and memory to a master, and starts executors when told to. It announces itself to the master on startup, which is the line step 4 looks for. A worker is long-lived; the executors it starts are not.
standalone mode	Spark running its own master and workers, rather than asking another system for resources. It is Spark's simplest arrangement and the one deployed here. It exists because Spark predates Kubernetes and had to schedule for itself.
Service	A stable name for whichever pods currently carry a label. The one new Kubernetes object this round adds. Pods are replaced and each replacement gets a new address, so nothing can be configured to talk to a pod directly. A Service is the name that does not change: workers are told <code>spark://spark-master:7077</code> once and that stays true across every replacement.
cluster DNS	The name server every pod is given, which answers for Service names. It is what turns <code>spark-master</code> into an address inside the cluster. It answers for pods that are behind a Service, and for nothing else , which is the trap panel 9 describes.

3 • DEPLOY THE MASTER AND THE WORKERS • STEP 2

Spark's own cluster is three things: a master process that hands out cores, worker processes that supply them, and a name the workers can dial that keeps working when a pod is replaced. In Kubernetes terms that is **two Deployments and one Service**.

`µLab 1` made a Deployment by typing `kubect1 create deployment whale ...`, one object per command. Three objects would be three commands, each with its own arguments to get right, and nothing written down afterwards. Instead they are **described in a file**, and the cluster is told to match it. Read it before you apply it.

```
kind: Service          name: spark-master
  clusterIP: None      # resolve the name to the pod itself
  selector: { app: spark-master } # whichever pod carries this label
  ports: 7077 (submit), 8080 (web)
```

```
kind: Deployment      name: spark-master    replicas: 1
  image: spark:3.5.4-python3
  command: spark-class org.apache.spark.deploy.master.Master --host spark-master
  hostname / subdomain: spark-master # so the pod has a name DNS answers for
  volumeMounts: /lab (hostPath /lab, read-only)
```

```
kind: Deployment      name: spark-worker    replicas: 2
  command: spark-class org.apache.spark.deploy.worker.Worker \
    spark://spark-master:7077 --cores 1 --memory 1g
  volumeMounts: /lab (hostPath /lab, read-only)
```

<code>kind</code>	What sort of object this is. The same word <code>kubect1 get</code> takes: <code>kubect1 get deployments</code> , <code>kubect1 get services</code> .
<code>clusterIP: None</code>	Do not give the Service an address of its own; make the name answer with the pod's address. The master binds to whatever its own name resolves to, and it cannot bind to a Service's address.
<code>selector</code>	The label a pod must carry to be behind this Service. Not a pod name: that is the whole point, since pod names change.
<code>command</code>	What to run instead of the image's default. <code>spark-class</code> starts one named Java class and stays in the foreground, which is what a container needs.

--host spark-master Tells the master which name to publish itself under. It must match what the workers dial, or they connect and are then sent somewhere they cannot reach.

hostname / subdomain Gives the pod a name of its own in cluster DNS. Without it the pod is reachable only through the Service, and panel 9 is what happens then.

--cores 1 --memory 1g What one worker offers. Deliberately small, so the arithmetic is legible: the cluster has as many cores as it has worker pods.

hostPath /lab Give the pod the folder the cluster container sees at /lab, which step 1 attached. Both Deployments get it: in μLab 3 the master runs the driver, which reads the script, and the workers run the executors, which read the corpus.

2 ☐ Apply it, then watch the pods appear.

In the `kubectl` shell. Nothing has been downloaded to your laptop: the file is read straight off the web, which is why there is no clone in this round.

```
kubectl apply -f https://raw.githubusercontent.com/oesteban/\
isc-302-2026-2027-spark-lab/main/spark-standalone.yaml
kubectl get pods
```

The address is one line, broken here only to fit the page. Type it without the backslash and without the line break.

apply Make the cluster match the description: create what is missing, change what differs, leave the rest alone. Safe to run twice.
 -f https://... -f is for *file*, and a URL counts as one: `kubectl` fetches it and reads it. `raw.githubusercontent.com` serves a file's contents rather than the web page around it.

Three objects are created, and it says so:

```
service/spark-master created
deployment.apps/spark-master created
deployment.apps/spark-worker created
```

The pods will sit in ContainerCreating for a while. One to three minutes here, and longer on a shared connection. Nothing is wrong: the cluster is downloading a 535 MB image, and panel 4 is about why it has to. Run `kubectl get pods` every so often until all three read 1/1 Running.

```
master pod _____ READY _____
worker pod _____ READY _____
worker pod _____ seconds until all three _____
```

4 • COMPARE THE CLUSTER'S IMAGE STORE WITH THE LAPTOP'S • STEP 3

The wait in step 2 was a download, and it is worth knowing what was downloaded and from where. The cluster runs its containers with containerd, not with your laptop's Docker, and the two keep entirely separate collections of images. Neither can read the other's. List both and compare.

3 ☐ List the images the cluster has, then the ones the laptop has.

Both lines are typed **on your laptop**, not in the `kubectl` shell. `crictl` exists only inside the cluster container, which is why the first line has to reach in there to run it.

```
docker exec k8s crictl images
docker image ls spark:3.5.4-python3
```

`docker exec k8s` Run a command **inside** the already-running `k8s` container, rather than starting a new one.
`crictl images` The cluster's own tool for talking to containerd, and its way of saying *list the images you can start*.
`docker image ls NAME` Docker's list, narrowed to one name. No rows at all is a perfectly good answer here.

As observed, with the laptop's copy present only because it had been pulled earlier:

IMAGE	TAG	IMAGE ID	SIZE
docker.io/library/spark	3.5.4-python3	5908cada5243a	535MB

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
spark	3.5.4-python3	5908cada5243	21 months ago	982MB

The `IMAGE ID` is the same in both, because it is the same image. The two `SIZE` columns disagree because they measure different things, the download and the unpacked copy, not because the images differ.

cluster IMAGE ID _____ laptop IMAGE ID _____

Now answer the first question.

If your laptop already had that image, the cluster downloaded a second copy of something that was sitting on the same disk. Say why it had to. Panel 2 defines **image store**, and the two listings above were produced by two different programs: read which command each one needed.

5 • READ THE MASTER'S LOG AND FIND THE WORKERS • STEP 4

Three pods running proves Kubernetes started three programs. It does not prove they found each other, and they are two separate things: Kubernetes knows nothing about Spark, and would report a worker shouting into the void as perfectly healthy. The master writes down every worker that reports in, so its log is where the answer is.

4 ☐ Read what the master has logged since it started.

In the `kubectl` shell, `deploy/spark-master` means *the pod of that Deployment*, so the pod's name does not have to be typed.

```
kubectl logs deploy/spark-master | grep Master:
```

logs	Print what a container has written to its output since it started. Nothing is running or changed by this.
deploy/spark-master	KIND / NAME. Given a Deployment, <code>kubectl</code> picks one of its pods and says which.
grep Master:	Ordinary shell again: keep only the lines the master class wrote, and drop several hundred lines of startup from every other part of Spark.

The three lines that matter, as observed:

```
INFO Master: Starting Spark master at spark://spark-master:7077
INFO Master: Registering worker 10.42.0.2:43581 with 1 cores, 1024.0 MiB RAM
INFO Master: Registering worker 10.42.0.3:39957 with 1 cores, 1024.0 MiB RAM
```

The address in the first line is the Service name from panel 2, not a pod. The other two are the workers announcing themselves, one line each, with what they are offering. Your addresses will differ; the shape will not.

workers registered _____ cores each _____
cores in total _____ RAM each _____

Now answer the question this round is about.

Two numbers now agree: the worker Deployment was asked for `replicas: 2` and the master reports two workers. They are counted by different things and they are not the same number. Say which one is which, and which of the two would be wrong first if a worker pod were deleted. Panel 2 defines both counters: what a **Spark master** keeps a list of, and what a **Service** stands in for. µLab 1 panel 9 is the other half, where a controller was shown replacing a deleted pod.

Solved means panels 1 to 5 are done: the lab folder unzipped and handed to the cluster, master and two workers all reading 1/1 Running, both image stores listed and compared, and the master's log naming both workers with the cores each one offered.

Reached this line? Then you have solved the challenge.



Referee: tick the box on the left now, and write the time. Do not wait for the caucus. Then carry on below — the extensions are not optional, they are the rest of the round.

Solved at _____ Referee's initials _____

6 • RUN A JOB ON THE CLUSTER YOU JUST BUILT

Everything so far shows Spark is **running**. This shows it **computes**. The job is the one that ships inside the image: throw darts at a square, count how many land inside a circle, and multiply. It needs no data, so nothing has to be copied anywhere, which makes it the cheapest possible proof that the master, the workers and a driver can all reach each other.

Typed in the `kubectl` shell. The driver runs **inside the master pod**, which looks like a detail and is not: panel 9 says what happens when it runs anywhere else.

```
kubectl exec deploy/spark-master -- /opt/spark/bin/spark-submit \
--master spark://spark-master:7077 \
--class org.apache.spark.examples.SparkPi \
local:///opt/spark/examples/jars/spark-examples_2.12-3.5.4.jar 100
```

exec	Run a command inside a container that is already running, the same idea as <code>docker exec</code> in step 3, one level in.
--	End of <code>kubectl</code> 's own arguments. Everything after it belongs to the command being run, so <code>--master</code> is read by Spark and not by <code>kubectl</code> .
spark-submit	Spark's launcher: start a driver, ask a master for cores, run the code, print the result, exit.
--master spark://...:7077	Which master to ask. The Service name from panel 2 and port 7077, which is the one the file labelled <i>submit</i> .

<pre>--class org.apache...SparkPi local: ///opt/spark/...jar</pre>	Which program inside the jar to run. A jar holds many; this names one. Where the jar is. <code>local:</code> means <i>already present in the image on every machine</i>, so Spark ships nothing. The third slash is the root of that filesystem. The program's own argument: how many slices to split the darts into. More slices, more pieces of work, and a more precise answer.
--	---

Several hundred lines scroll past and one line matters. On a two-worker cluster the whole thing took **8 seconds**:

Pi is roughly 3.1406367140636715

The digits after the third will not match anyone else's, including a second run of your own, because the darts are thrown at random. Anything near 3.14 is a success.

your value _____ seconds _____

7 • DELETE A WORKER POD AND WATCH THE MASTER NOTICE

µLab 1 deleted a pod and watched a controller put it back. The same thing happens here, except that something else was relying on that pod, so there are now two reactions to watch instead of one, written down by two different programs.

```
kubectll get pods
kubectll delete pod <one of the two workers>
kubectll get pods
kubectll logs deploy/spark-master | grep Master: | tail -3
```

<pre>delete pod tail -3</pre>	Remove one pod. It removes nothing else: the Deployment that asked for it is untouched and still wants two. Shell: keep the last three lines, which are now the three most recent things the master wrote.
---------------------------------	--

As observed, in this order:

```
INFO Master: Removing worker worker-...-10.42.0.5-33537 on 10.42.0.5:33537
INFO Master: Telling app of lost worker: worker-...-10.42.0.5-33537
INFO Master: Registering worker 10.42.0.9:40593 with 1 cores, 1024.0 MiB RAM
```

The replacement pod starts in seconds rather than minutes, because the image is now in the cluster's store. Write down the new pod's name and the new address in the master's log, then say which of the two put the worker back, the Kubernetes controller or the Spark master, and what the other one did instead.

8 • EXTRA MILE • HAND THE CLUSTER THE IMAGE INSTEAD OF LETTING IT DOWNLOAD IT

Step 2 waited while the cluster downloaded 535 MB that may already have been on the same disk. It does not have to: an image can be written out of one store and read into the other, over a pipe, with no network at all. Time it, and compare with the wait you recorded in step 2.

On your laptop. If `docker image ls` found nothing in panel 4, this needs `docker pull spark:3.5.4-python3` first, which is the download you are trying to avoid, so the payoff is for the **next** cluster rather than this one.

```
docker save spark:3.5.4-python3 | docker exec -i k8s ctr -n k8s.io images import -
```

<pre>docker save NAME docker exec -i k8s ctr -n k8s.io images import -</pre>	Write an image out as a plain stream of bytes. With no <code>-o</code> file it goes to standard output. Ordinary shell, nothing to do with Docker: send the left command's output straight into the right command's input, without ever writing a 1 GB file to disk. <code>-i</code> keeps the container's input open, which is what the pipe needs. There is no <code>-t</code> because nothing here is typed by hand. containerd's own tool, and the namespace Kubernetes reads. An image imported into any other namespace is invisible to it. Read an image stream and add it to the store. The <code>-</code> is the other end of the pipe.
--	--

seconds to import _____ seconds to download, step 2 _____

Sixteen laptops downloading the same 535 MB at once is the slowest thing in this room. Say what you would do differently if you were setting this lab up for a class.

9 • ANNEX • WHY THE DRIVER HAS TO RUN SOMEWHERE THAT HAS A NAME

Panel 6 submitted the job from inside the master pod. Submitting from a pod of your own looks tidier and does not work. Executors are told to call the driver back by name, and cluster DNS answers only for pods that are behind a Service. A pod started on its own has no name anyone can resolve, so every executor exits immediately and the driver waits for cores that never arrive, printing this every fifteen seconds:

```
WARN TaskSchedulerImpl: Initial job has not accepted any resources; check your
cluster UI to ensure that workers are registered and have sufficient resources
```

The message blames the workers, and the workers are fine. This is also why the Deployment in panel 3 sets `hostname` and `subdomain`: built with `kubectrl create deployment` instead, the master pod would be called `spark-master-fccf6ddcd-dmf5m` and every submission would hang in exactly this way. That was measured, not predicted.

What production does instead. Standalone mode is Spark scheduling for itself, from before Kubernetes existed. Given a real cluster, a submission made with `--master k8s://...` skips all of this: Kubernetes is asked directly for a driver pod and executor pods, they exist only while the job runs, and there is no master, no workers and nothing standing idle in between. It is also several more things to get right, which is why it is here and not in step 2.

Done when the referee can say, without reading this sheet, why the cluster downloaded an image the laptop already had, and what a Service is for.